Calyr Architecture

Rupert Tscheliessnig

29 April 2026

The Problem

Scientific systems today are fragmented:

- Data systems mixed with logic
- Models tied to applications
- Workflows locked in scripts
- No clear separation of responsibilities

Result

extbf*imes* No scalability
extbf*imes* No reuse
extbf*imes* No clean product boundaries
extbf*imes* Future technical chaos

The Solution

Separate the system into three distinct layers

PLATFORM

Calyr
Runs everything. Orchest rates workflows. Manages data.

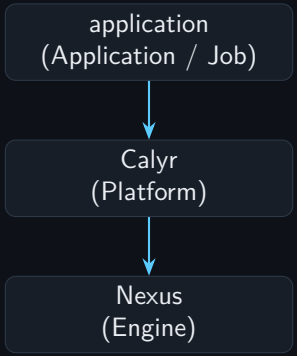
ENGINE

Nexus
Does the thinking. Processes data. Portable.

APPLICATION

application
Uses Nexus. Solves problems. Produces insights.

Core Architecture



- application depends on Calyr
- Calyr orchestrates Nexus and application
- Nexus does NOT depend on Calyr (portable)

Calyr: Platform

Role

Calyr is the operational platform that runs jobs, orchestrates workflows, and manages all data infrastructure.

Calyr does:

- Runs jobs
- Schedules workflows
- Manages users and access
- Stores raw data
- Handles infrastructure

Calyr does NOT:

- Define scientific meaning
- Contain domain logic
- Interpret results
- Build model knowledge

Key principle

Calyr is the framework. It does NOT know about SAXS, SPR, or any specific science.

Nexus: Engine

Role

Nexus is the thinking engine that processes data, builds states, and generates scientific understanding.

Nexus does:

- Processes data
- Builds structural states
- Links structure, kinetics, thermodynamics
- Generates hypotheses
- Evaluates models

Nexus key property:

- **Portable**
- Can run without Calyr
- Can be embedded in other systems
- Reusable across applications

Key principle

The Application Layer

Role

The application layer is domain-specific that uses Nexus to solve real scientific problems.

Application does:

- Uses Nexus for analysis
- Solves domain-specific problems
- Produces scientific insights
- Integrates SAXS + SPR

Application represents:

- Application layer
- Domain expertise
- User-facing layer
- Use-case implementation

Key principle

Application depends on Nexus, not the other way around. Replaceable. Extensible.

Execution Flow

User $\Rightarrow$ **application** $\Rightarrow$ **Calyr** $\Rightarrow$ **Nexus** $\Rightarrow$ **Result**

- User provides experiment or analysis request to **application**
- application formulates it as a job and submits to **Calyr**
- Calyr schedules and runs the job using **Nexus** as the compute engine
- Nexus processes data, builds models, produces output
- Result flows back through the stack to the user

Data Architecture

Two distinct data warehouses

Calyr Warehouse

- Raw experimental data
- Datasets from instruments
- FAIR metadata
- Provenance tracking
- User files and projects

(Storage layer)

Nexus Warehouse

- Extracted features
- Structural states
- Model outputs
- Computed observables
- Analysis results

(Understanding layer)

Key insight

Data flows from Calyr storage through Nexus understanding to produce scientific knowledge

Separation of Concerns

Calyr
moves data

Nexus
understands data

application
uses the understanding

Each component has one job. Each does it independently. Together they solve the problem.

Dependency Rule

- **application depends on Nexus** — application needs the engine
- **Calyr orchestrates both** — platform runs everything
- **Nexus does NOT depend on Calyr** — engine is independent and reusable

Why This Matters

For developers:

- Nexus can be tested independently
- Clear contracts between layers
- Easy to replace implementations
- Simpler debugging

For the business:

- Modular, scalable architecture
- Reusable engine (Nexus) across projects
- Independent applications
- Clear product boundaries

Clean scalability. Investor-ready. Future-proof.

Final Statement

Calyr
runs the system

Nexus
defines the system

application
applies the system

No ambiguity. Clean architecture. Publishable.